Project Proposal

UniClass AI: A University Learning Management System with an Explainable AI Student-Risk Advisor

Team CSE4104-7E-T04 | Section 7E | Course: CSE4104

Team Name	CSE4104-7E-T04
Section	7E
Team Members	Sakib Hasan Anik (CSE230321575), Team Leader Arnab Basu (CSE230321361) Sarder Fariha Alam Trishna (CSE230321371)
GitHub Repository	https://github.com/SakibHasanAnik/Uniclass-AI
Submitted To	Md. Riaz Mahmud, Assistant Professor, Dept. of CSE

1. Project Title

UniClass AI: A University Learning Management System with an Explainable AI Student-Risk Advisor

UniClass AI is a web platform where teachers manage courses, materials, assignments, marks and attendance, and students follow their own progress. On top of this, the system gives academic advisors a weekly list of students who may fail or drop out. For each flagged student it explains the reasons in plain language, suggests an action, and drafts a supportive message that the advisor can edit and send. Since all the learning data is already collected inside the platform, the AI part does not need any extra data entry from teachers.

2. Problem Statement

2.1 The current problem

In our university, information about a student's progress is spread across many places. Course materials are shared through a general classroom tool, marks are kept in each teacher's spreadsheet, and attendance is usually taken on paper or in a separate sheet. A teacher only sees their own course, while an academic advisor is responsible for a large group of students taking many different courses. Because of this, the early signs that a student is struggling, such as falling attendance, missed assignments or quiz marks far below the class, are scattered across several teachers and files. In most cases nobody sees the full picture until the semester results are published, and by then it is too late to help.

2.2 Why the problem is important

- **Failing is costly for students.** A failed course means repeating it, paying extra tuition and graduating late. In the worst case the student leaves university.
- **Early help works much better.** A meeting, tutoring or a lighter course load in week 5 can change the result; a warning after the final exam cannot.
- **Advisors do not have enough time.** With many advisees each, they need to know whom to contact first and what to talk about.

2.3 Why existing solutions are not enough

Existing solution	Limitation
Google Classroom and similar tools	Good for materials and assignments, but there is no attendance module, no combined view of marks across courses and no role for academic advisors.
Moodle-type LMS	Has many features but is heavy to set up. Its reports describe what already happened; they do not predict which students are likely to struggle.
Spreadsheets and paper	No single source of data, no automatic calculation, and it is not practical to analyse hundreds of students by hand.
Commercial early-warning systems	Expensive, designed for other countries' institutions, and often give a score without a clear reason behind it.

2.4 Why AI is relevant

Identifying a struggling student means looking at several small signals together. A slight drop in attendance alone may mean nothing, but together with missed assignments and falling marks it is a strong warning. A machine learning model can learn how these signals combine and apply the same judgement to every student each week, which no advisor can do manually for an entire department. If the model is explainable, each prediction can be turned into readable reasons that the advisor can check and trust. A large language model (LLM) can then save time by drafting a kind, personal message. The AI does not replace the advisor; it shows them where to look first.

3. Objectives

1. Build one platform where courses, materials, assignments, marks and attendance are managed and stored together.
2. Provide separate portals for students, teachers, advisors and administrators, where each user sees only the data their role allows.
3. Train a machine learning model that gives each student a weekly risk score and level (High, Medium or Low) based on attendance, marks and submission behaviour.
4. Explain every prediction with the top two or three reasons, written in plain language with real numbers.
5. Suggest an intervention for each flagged student and use an LLM to draft an editable message to that student.
6. Allow advisors to record the actions they take and the outcome, so that follow-up can be tracked.
7. Evaluate the model properly using recall, precision and a simple rule-based baseline, and report its limits and ethical safeguards.

4. Proposed Features

Module	Features
Authentication	Login page with Student, Teacher and Advisor options; a separate admin login page (/adminlogin); token-based login; role check on every request; logout.
Student portal	Dashboard with enrolled courses, attendance percentage, average marks and upcoming deadlines; course page with materials, assignment submission, results compared with the class average, and attendance history. Students do not see risk scores.
Teacher portal	Dashboard with courses, total students, submissions to grade and today's classes; upload materials; create and grade assignments; create quizzes, class tests and midterms; enter marks with validation; take attendance; class analytics; list of students needing attention.
Advisor portal	Risk dashboard of the advisor's own students with colour-coded levels and filters; student profile with AI reasons, course performance and risk trend; suggested action; AI-drafted message; intervention log.
Admin console	Manage users of all roles; create courses with teacher and class schedule; enroll students; assign students to advisors; run AI scoring and view model results.
AI engine	Feature extraction, risk prediction, explanation, action recommendation and LLM message drafting (details in Section 5).
Analytics	Charts for marks trends, score distribution, weekly attendance and the distribution of risk levels.
Notifications	In-app indicators for upcoming deadlines and new submissions to grade.

5. AI Integration

AI is the main part of this project rather than an extra feature. It has two parts: a predictive model that decides which students need attention, and an LLM that helps the advisor act on that information.

5.1 Risk prediction

Every week the system creates one row of features for each student, using data already stored in the database: attendance rate and its recent trend, consecutive absences, average marks, gap from the class average, marks trend, ratio of missed and late assignments, previous CGPA and course load. A logistic regression model (scikit-learn) converts these features into a risk score from 0 to 100, which is mapped to High (60 and above), Medium (30 to 59) or Low (below 30). Before the model, we will build a simple rule-based score, which serves both as a baseline for comparison and as a fallback.

5.2 Explainability

For each student we calculate how much every feature contributed to the score (the learned weight multiplied by how far the student is from the average). The largest contributions are

converted into sentences such as "4 of 6 assignments not submitted" or "Marks 18 points below class average". For a linear model these reasons are exact, so an advisor can check them against the raw data.

5.3 Suggested action and AI message

The main reason for a student's risk is mapped to a suggested action: attendance problems lead to a personal check-in, low marks to a tutoring referral, and missed work to a deadline-planning session. When the advisor clicks "Generate with AI", only the reasons and the action are sent to the Groq LLM API. The student's name and ID are never sent. The API returns a short, supportive message, which the advisor edits and sends personally. If the API is not available, the system shows a template message instead.

5.4 Why we chose this design

- **Our own trained model makes the decision.** It is reproducible, can be tested with standard metrics and is easy to explain. An LLM used alone would give inconsistent scores and reasons we cannot verify.
- **The LLM is only used for writing the message**, which is the task it does best.
- **A human stays in control.** The score only starts a conversation; it never leads to a penalty or an automatic message.

5.5 Data, evaluation and ethics

- **Data:** we will practise on the public UCI dataset "Predict Students' Dropout and Academic Success", then train the system on realistic synthetic data generated in our own database structure. Real student records would need university approval.
- **Evaluation:** an 80/20 train-test split; recall (target of at least 75%), precision, F1 score and a confusion matrix, compared with the rule-based baseline.
- **Ethics:** sensitive attributes such as gender, religion or income are not used; students and teachers cannot see risk scores; advisors only see their own students; no personal data is sent to the LLM.

6. Technology Stack

Layer	Technology	Purpose
Frontend	React.js (Vite), JavaScript, HTML, CSS, Tailwind CSS, React Router, Axios, Recharts	Role-based web interface, dashboards and charts
Backend	Python, Django, Django REST Framework	REST API, login and role-based access, validation, file uploads
Database	PostgreSQL	Storing users, courses, marks, attendance and AI results
AI / ML	pandas, NumPy, scikit-learn (logistic regression), joblib	Feature extraction, training, prediction and evaluation
AI Integration	Groq API (LLM, free tier)	Drafting messages for advisors

Layer	Technology	Purpose
Deployment	Vercel (frontend), Render (backend and database)	Online demo of the system
Tools	Git, GitHub, VS Code, Postman, Jupyter Notebook	Version control, development, API testing and experiments

6.1 How the parts work together

The React frontend runs in the browser and sends requests to the Django REST API with the user's login token. Django checks the user's role, reads or writes data in PostgreSQL through its ORM, and returns JSON. The AI module is a separate Python package that runs once a week (or when the admin clicks "Run now"); it reads marks, attendance and submissions, and saves a new risk score for each student. The advisor dashboard simply reads these saved results. The Groq API is called only when an advisor asks for a message.

7. Expected Outcome

7.1 What users will gain

User	Benefit
Students	One place for materials, deadlines, marks and attendance; a clear view of their standing compared to the class; earlier support from their advisor.
Teachers	Faster marks and attendance entry, automatic class statistics, and an instant list of students who need attention.
Advisors	A ranked list of whom to contact each week, with reasons, a suggested action and a ready-to-edit message.
Administration	Central management of users and courses and a department-wide view of how many students are at risk.

7.2 How the workflow improves

Task	Current workflow	With UniClass AI
Finding struggling students	Noticed after final results, or by chance	Flagged every week from around week 4, with reasons
Collecting course data	Advisor asks each teacher or checks several spreadsheets	All courses, marks and attendance on one profile
Contacting the student	Advisor writes each message from scratch	AI drafts the message; advisor edits and sends it
Tracking the help given	Informal or not recorded	Intervention log with outcome and follow-up date
Class statistics	Manual spreadsheet formulas	Automatic averages, distributions and charts

7.3 Expected benefits of AI integration

- **Earlier help:** at-risk students are found weeks before the results, while there is still time to improve.
- **Consistency:** every student is checked using the same criteria every week.
- **Trust:** readable reasons let advisors verify each warning instead of following a black box.
- **Time saved:** advisors spend their time talking to students instead of collecting data.

8. Project Plan Summary

The project will be completed in 14 weeks. A detailed week-by-week plan is given in the separate Initial Project Planning document.

Weeks	Phase	Main deliverables
1 - 2	Planning and design	Proposal, UI designs, ER diagram, API list, GitHub repository
3 - 4	Foundation	Database models, demo data, login and admin login, page layout
5 - 7	Core LMS	Courses, materials, assignments, marks and attendance for teachers and students
8 - 11	AI development and integration	Feature extraction, model training, risk scoring, advisor portal, AI message
12 - 13	Testing and deployment	Integration and role-access testing, bug fixes, online deployment
14	Final delivery	Final report, presentation and live demo

9. Conclusion

UniClass AI addresses a real problem in our university: struggling students are noticed too late because their information is scattered and advisors do not have enough time. By combining a practical learning management system with an explainable risk model and an LLM-assisted advising workflow, the project uses AI where it adds real value: finding patterns across many students, explaining them clearly, and helping advisors act early. The scope is realistic for a three-member team in one semester, and the final result will be a working system with an honest evaluation of its AI component.